

CloudShield AI / DAIL

15 — Implementation Roadmap & Definition of Done

Version 1.0 | Final Pre-Implementation Baseline | 1 September 2026

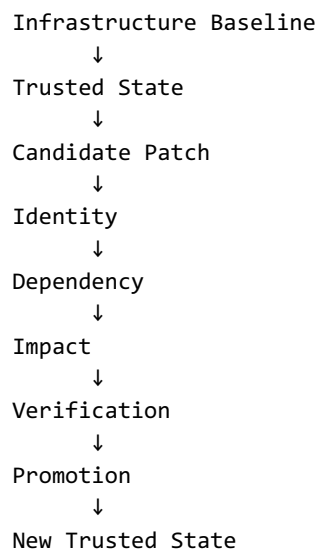
Purpose: Convert Specifications 01–14 into one executable implementation plan. This document defines implementation order, workstreams, dependencies, milestones, integration gates, demonstration checkpoints, deliverables, Definition of Done, release gates, and the exact transition from documentation to building the DAIL system.

Document Control

Field	Value
Document ID	CSAI-DAIL-ROADMAP-015
Status	FINAL PRE-IMPLEMENTATION BASELINE
Source specifications	01–14 DAIL engineering specification set
Primary output	Implemented, tested, deployable DAIL MVP + experiment evidence
Planning principle	Build from deterministic foundations upward; integrate LLM last; never bypass safety gates.
Completion principle	A phase is complete only when its stated artifacts and verification gates pass.

1. Implementation Objective

Build a working CloudShield AI / DAIL MVP that can ingest a versioned infrastructure baseline, represent Trusted State, accept an AI-generated repair candidate, identify changed resources and dependencies, compute impact, verify protected security and functional invariants, persist evidence, and promote only candidates that satisfy all required gates.

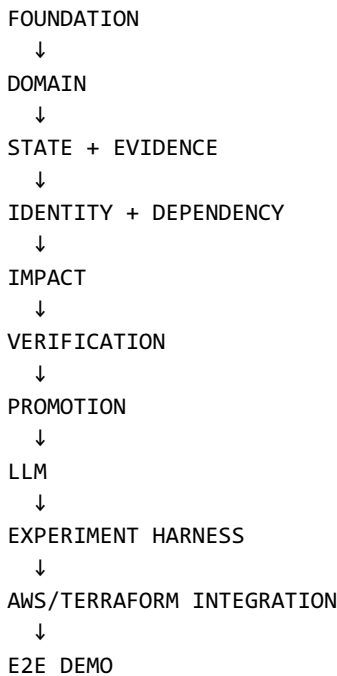


2. Final MVP Outcome

- A reproducible repository and development environment.
- Terraform/AWS fixture set.
- Canonical domain/state model.
- Identity and dependency engine.
- Impact/invalidation engine.
- Deterministic verification engine.
- Promotion controller with stale-parent protection.
- Evidence and structured logging subsystem.
- LLM adapter with strict candidate-only boundary.
- Experiment harness and metrics.
- Automated QA suite and CI pipeline.
- End-to-end demonstration of safe promotion and unsafe-candidate rejection.

3. Implementation Strategy

Implementation proceeds from the bottom of the trust stack upward. Deterministic state, invariants, evidence, identity, dependency, impact, and verification are implemented before LLM generation is allowed to participate in the end-to-end path.

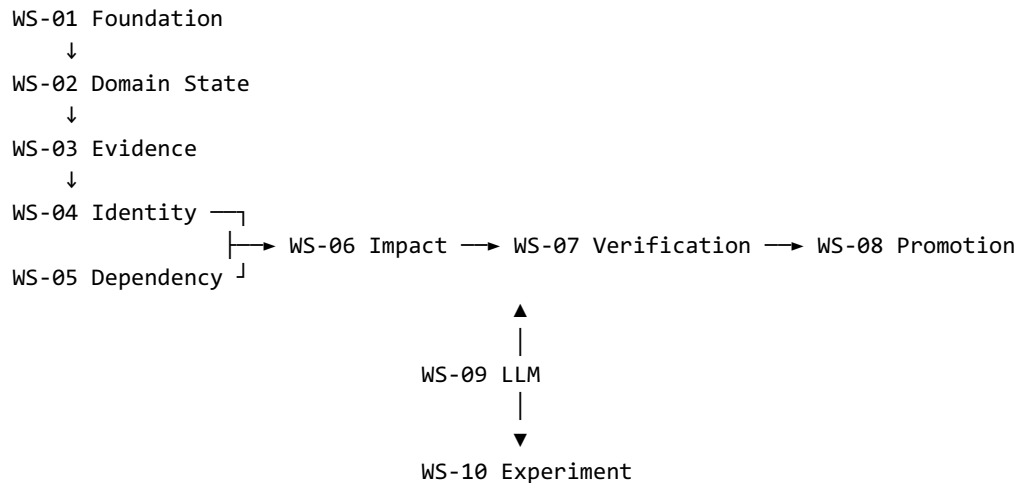


4. Workstreams

Workstream	Primary responsibility
WS-01 Foundation	Repository, runtime, configuration, database/storage foundations.
WS-02 Domain State	State, candidate, invariant, lifecycle models.
WS-03 Evidence	Evidence records, audit events, hashes, lineage.
WS-04 Identity	Resource matching and change classification.

WS-05 Dependency	Typed dependency graph.
WS-06 Impact	Invariant impact and invalidation.
WS-07 Verification	Deterministic security/function predicates.
WS-08 Promotion	Decision engine and atomic Trusted State transition.
WS-09 LLM	Provider adapter, prompts, structured output, patch validation.
WS-10 Experiment	Trials, baselines, metrics, reports.
WS-11 Infrastructure	Terraform/AWS sandbox and deployment automation.
WS-12 QA/DevOps	CI, security checks, regression suite, release automation.

5. Dependency Graph



WS-11 Infrastructure + WS-12 QA/DevOps span all stages.

6. Phase Overview

Phase	Focus	Exit gate
P0	Project bootstrap	Repository and CI healthy.
P1	Domain foundation	State/invariant contracts tested.
P2	Evidence foundation	Evidence/audit lineage works.
P3	Identity + dependency	Deterministic graph/change analysis works.
P4	Impact + invalidation	Affected obligations identified correctly.
P5	Verification	Security/function predicates produce correct outcomes.
P6	Promotion	Safe atomic state transition works.
P7	LLM integration	Untrusted model output becomes validated candidate only.
P8	Experiment harness	Controlled experiments and metrics work.
P9	AWS/Terraform integration	Sandbox deployment/integration works.
P10	Hardening + E2E	Full demo and release gates pass.

7. P0 — Project Bootstrap

7.1 Tasks

1. Create repository structure.
2. Pin runtime and dependencies.
3. Configure formatter/linter/type checks.
4. Configure automated tests.
5. Create configuration schema.
6. Create local database/storage setup.
7. Create CI workflow.
8. Create documentation index linking Specifications 01–15.

7.2 Deliverables

- Repository skeleton.
- Lockfiles/version pins.
- Local bootstrap command.
- Initial CI pipeline.
- Base test suite.
- Environment configuration templates.

7.3 Exit Criteria

- Fresh clone can bootstrap.
- CI runs successfully.
- Unit test command works.
- No secrets required for core tests.
- Main branch protected.

8. P1 — Domain Foundation

8.1 Tasks

- Implement Resource model.
- Implement Trusted State.
- Implement Candidate State.
- Implement Invariant model/registry.
- Implement lifecycle states.
- Implement canonical serialization/hashing.
- Implement state version/parent semantics.

8.2 Core Contracts

```
TrustedState {  
  state_id  
  version  
  content_hash  
  invariant_registry_version  
  created_at  
  parent_state_id | null  
}
```

```
CandidateState {  
  candidate_id  
  parent_state_id  
  patch_hash  
  status  
}
```

8.3 Exit Criteria

- Domain schemas validate.
- State lineage tests pass.
- Canonical hashes are stable.
- Forbidden lifecycle transitions are rejected.

9. P2 — Evidence Foundation

9.1 Tasks

- Implement EvidenceRecord.
- Implement AuditEvent.
- Implement structured LogEvent.
- Implement correlation IDs.
- Implement content hashing.
- Implement validity/invalidation transitions.
- Implement artifact references.
- Implement secret redaction.

9.2 Exit Criteria

- Evidence persists and resolves.
- Hashes verify.
- Lineage queries work.
- Invalidation preserves history.
- Redaction tests pass.
- Duplicate events are handled.

10. P3 — Identity & Dependency

10.1 Identity

- Implement canonical resource identity.
- Implement matching rules.
- Implement CREATED/MODIFIED/DELETED/UNCHANGED/REPLACED/UNCERTAIN outcomes.
- Implement provenance.

10.2 Dependency

- Implement graph model.
- Implement supported Terraform/AWS edge types.
- Implement unknown/unsupported states.

- Implement deterministic graph serialization.
- Implement cycle-safe traversal.

10.3 Exit Criteria

- Canonical fixtures pass.
- Ambiguity is conservative.
- Graph output is deterministic.
- No silent missing dependencies.

11. P4 — Impact & Invalidation

11.1 Tasks

- Build changed-resource set.
- Traverse dependency graph.
- Map changes to invariant work set.
- Generate invalidation events.
- Handle uncertainty/unknown dependencies.
- Generate ImpactReport.

11.2 Exit Criteria

- Direct impacts detected.
- Transitive impacts detected.
- Unrelated changes remain unaffected.
- Unknown/unsupported cases are conservative.
- Impact precision/recall fixtures pass.

12. P5 — Verification

12.1 Tasks

- Implement security invariant predicate.
- Implement functional EC2—RDS TCP/5432 predicate.
- Implement verification work set.
- Implement PASS/FAIL/UNKNOWN/UNSUPPORTED/VERIFIER_ERROR.
- Persist VerificationEvidence.
- Implement result aggregation.

12.2 Exit Criteria

- Known PASS fixtures pass.
- Known FAIL fixtures fail.
- Unknown/unsupported cases do not become PASS.
- Verifier errors fail closed.
- Candidate/state hashes match.

13. P6 — Promotion

13.1 Tasks

- Implement PromotionDecision.
- Implement required-check policy.
- Implement stale-parent protection.
- Implement atomic Trusted State commit.
- Implement idempotency/concurrency control.
- Persist promotion evidence.

13.2 Mandatory Scenarios

Scenario	Expected
SEC PASS + FUNC PASS	PROMOTE.
SEC PASS + FUNC FAIL	REJECT.
SEC FAIL + FUNC PASS	REJECT.
UNKNOWN required check	No promotion.
VERIFIER_ERROR	No promotion.
Stale parent	No promotion.
Concurrent promotion	Only valid current parent may advance.
Duplicate request	Idempotent.

13.3 Exit Criteria

No unsafe candidate can advance Trusted State, and a valid candidate can advance exactly once through the transaction boundary.

14. P7 — LLM Integration

14.1 Tasks

- Implement provider-neutral LLM interface.
- Implement prompt/version management.
- Implement context builder.
- Implement token budget estimator.
- Implement structured response parser.
- Implement HCL patch validator.
- Implement retry policy.
- Implement provider error normalization.
- Implement request/response evidence.

14.2 Exit Criteria

- Malformed output rejected.
- Forbidden operations rejected.
- Prompt injection fixtures pass.
- LLM confidence cannot promote.
- LLM cannot mutate Trusted State.
- Valid patch reaches Candidate State only.

15. P8 — Experiment Harness

15.1 Tasks

- Implement Experiment definition.
- Implement Trial lifecycle.
- Implement condition/baseline models.
- Implement repetition/seed recording.
- Implement metric extraction.
- Implement experiment reports.
- Implement artifact directory/indexing.
- Implement replay/rerun semantics.

15.2 Initial Experiment

Conditions:

C1 = No Repair
C2 = Deterministic Repair
C3 = LLM-only
C4 = LLM + DAIL

Core fixtures:

public SSH violation
safe functional path
security repair breaks function
safe repair

15.3 Exit Criteria

- Trials are reproducible.
- All compared conditions share compatible fixtures.
- Metrics have frozen definitions.
- Failed/excluded trials remain visible.
- Reports derive from persisted evidence.

16. P9 — Terraform & AWS Integration

16.1 Tasks

- Implement Terraform modules/environment roots.
- Provision controlled sandbox.
- Create canonical EC2/RDS/security-group topology.
- Implement state/configuration ingestion.
- Integrate AWS observations required by verification where defined.
- Secure Terraform state.
- Configure deployment roles.

16.2 Exit Criteria

- Terraform fmt/validate pass.
- Sandbox provisions successfully.
- Canonical infrastructure scenario is reproducible.

- No production credentials required.
- AWS integration failures fail safely.

17. P10 — Hardening & End-to-End

17.1 Tasks

- Run complete regression suite.
- Run adversarial suite.
- Run fault injection.
- Run concurrency tests.
- Run evidence integrity tests.
- Run full experiment.
- Run E2E safe promotion.
- Run E2E unsafe rejection.
- Run deployment smoke tests.
- Produce final QA and experiment reports.

17.2 Exit Criteria

- All mandatory release gates pass.
- No S0/S1 defects open.
- End-to-end evidence chain complete.
- Demo is repeatable from clean environment.

18. Exact Implementation Order

9. Repository/bootstrap.
10. Configuration and persistence foundations.
11. Domain/state/invariant contracts.
12. Evidence/audit infrastructure.
13. Canonical Terraform fixture representation.
14. Identity engine.
15. Dependency graph.
16. Impact/invalidation engine.
17. Verification engine.
18. Promotion transaction.
19. LLM adapter and candidate generation.
20. Experiment harness.
21. AWS/Terraform sandbox integration.
22. Full QA/hardening.
23. Final end-to-end demonstration.

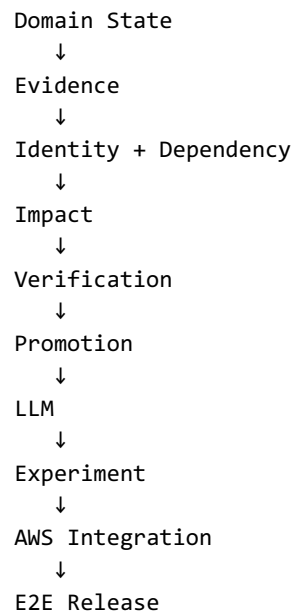
Do not start with the LLM. The LLM is intentionally late in the sequence because the deterministic trust pipeline must exist before AI-generated candidates are introduced.

19. Parallel Work Opportunities

Can proceed in parallel	Dependency
-------------------------	------------

CI/DevOps	Can start during P0 and continue throughout.
Terraform fixtures	Can start during P1/P2.
Evidence schema	Can progress alongside domain models.
Test fixture creation	Can begin before corresponding implementation.
LLM prompt prototypes	Can be prepared during P5 but integrated only after promotion exists.
Experiment report templates	Can be prepared during P6/P7.

20. Critical Path



The critical path is governed by safety dependencies rather than UI/demo convenience.

21. Milestone M1 — Foundation Complete

- Repository operational.
- Domain schemas implemented.
- Initial database/storage operational.
- CI green.
- Basic fixture loading works.

No AI integration required.

22. Milestone M2 — Deterministic DAIL Core Complete

- Identity works.
- Dependency graph works.
- Impact works.
- Verification works.
- Evidence works.

At M2, DAIL must be able to analyze a candidate without an LLM.

23. Milestone M3 — Trust Transition Complete

- Promotion controller works.
- Safe candidate promotes.
- Unsafe candidate rejects.
- Stale candidate rejects.
- Audit/evidence chain complete.

24. Milestone M4 — AI Integration Complete

- LLM generates structured candidate.
- Patch validation works.
- LLM failures are isolated.
- Candidate enters deterministic DAIL pipeline.
- No model output can bypass promotion.

25. Milestone M5 — Research Harness Complete

- Experiments run reproducibly.
- Baselines work.
- Metrics generated.
- Trial artifacts linked.
- Comparison report generated.

26. Milestone M6 — AWS Demonstration Complete

- Terraform sandbox works.
- Canonical AWS topology exists.
- DAIL can process realistic infrastructure context.
- Verification evidence can be demonstrated.
- Full safe/unsafe lifecycle is repeatable.

27. Definition of Done — Documentation

- Specifications 01–15 are synchronized with implementation.
- Changed contracts documented.
- Version references correct.
- Known limitations documented.

28. Definition of Done — Code

- Implementation complete.
- Types/contracts validated.
- No dead placeholder paths in production flow.
- Error handling explicit.
- No unsafe fallback behavior.
- Core algorithms deterministic where required.

29. Definition of Done — Tests

- Unit tests pass.
- Contract tests pass.
- Integration tests pass.
- Adversarial tests pass.
- Regression tests pass.
- Safety lifecycle tests pass.
- Concurrency/fault tests pass where applicable.
- CI is green.

30. Definition of Done — Security

- No committed secrets.
- Secret scanning passes.
- LLM prompt redaction passes.
- IAM permissions reviewed.
- Terraform state protected.
- Trust boundary tests pass.
- Tampering/integrity tests pass.

31. Definition of Done — Evidence

- Every material decision has evidence.
- Hashes verify.
- State/candidate lineage resolves.
- Promotion decision references verification evidence.
- Failed attempts remain auditable.
- Artifacts are reproducibly addressable.

32. Definition of Done — LLM

- Model/provider configuration recorded.
- Prompt/context versions recorded.
- Structured output validated.
- Patch scope validated.
- Retry bounded.
- Provider failures normalized.
- LLM cannot promote directly.

33. Definition of Done — Experiment

- Experiment protocol frozen.
- Fixture versions frozen.
- Baselines declared.
- Trial metadata complete.
- Metrics definitions frozen.

- Results reproducible.
- Excluded failures explicitly classified.

34. Definition of Done — Infrastructure

- Terraform validates.
- Plan reviewed.
- Environment isolated.
- Deployment identity authorized.
- Smoke tests pass.
- Deployment metadata recorded.
- Rollback path validated.

35. Final System Definition of Done

DAIL MVP = DONE when:

```

Input State
↓
Candidate Generation
↓
Identity
↓
Dependency
↓
Impact
↓
Verification
↓
Evidence
↓
Promotion
↓
Trusted State

```

works end-to-end,
is reproducible,
is tested,
and cannot bypass its safety gates.

- Canonical unsafe-repair scenario is rejected.
- Canonical safe-repair scenario is promoted.
- All mandatory evidence is persisted.
- Stale parent is rejected.
- LLM malformed output is rejected.
- CI/release gates pass.
- Experiment harness produces a valid report.
- AWS/Terraform sandbox demo is repeatable.

36. Release Blocking Conditions

- Unsafe promotion observed.
- Required verifier returns incorrect PASS.
- Promotion bypass discovered.
- Evidence integrity failure affecting required proof.
- Stale candidate accepted.
- Secret leakage.
- Critical state corruption.
- S0/S1 unresolved defect.
- Core regression suite failure.

37. Demo Definition

The final demonstration should be one deterministic story that shows the safety property rather than merely showing an AI-generated Terraform patch.

24. Start from a known baseline.
25. Show the protected security and functional invariants.
26. Generate a candidate repair.
27. Show changed resources.
28. Show dependency/impact analysis.
29. Show verification results.
30. Demonstrate rejection of a candidate that breaks the functional invariant.
31. Generate/execute a safe candidate.
32. Show both required invariants PASS.
33. Promote candidate.
34. Show new Trusted State and complete evidence lineage.

38. Final Research Deliverables

- Working DAIL software.
- Versioned Terraform/AWS fixtures.
- Automated QA suite.
- Experiment harness.
- Baseline implementations.
- Experiment reports.
- Evidence/audit artifacts.
- Architecture/specification set.
- Deployment configuration.
- End-to-end demonstration.

39. Project Completion Review

Area	Completion test
Architecture	Implementation matches Specifications 01–15.
Domain	State/invariant lifecycle correct.
Analysis	Identity/dependency/impact correct.
Safety	Verification and promotion fail closed.

AI	LLM isolated as candidate generator.
Evidence	Lineage/integrity complete.
QA	Mandatory suite green.
Research	Experiment results reproducible.
Infrastructure	Terraform/AWS sandbox repeatable.
Operations	CI/CD and recovery documented.

40. Post-MVP Backlog

- Additional AWS services.
- More invariant families.
- More dependency types.
- Advanced LLM model comparison.
- Human approval workflows.
- Production-scale observability.
- Cost optimization.
- Advanced statistical analysis.
- Additional provider adapters.
- Continuous environment monitoring.

Post-MVP work must not be introduced into the core MVP critical path unless it is required to satisfy a defined acceptance criterion.

41. Implementation Rules We Must Not Break

- Do not begin by vibe-coding the whole application.
- Do not make the LLM the center of the architecture.
- Do not implement promotion before verification.
- Do not implement verification without explicit invariants.
- Do not implement impact without identity/dependency semantics.
- Do not allow logs to substitute for evidence.
- Do not allow model confidence to substitute for verification.
- Do not use production infrastructure for early development.
- Do not silently ignore UNKNOWN or UNSUPPORTED results.
- Do not rewrite Trusted State history to make demos look successful.

42. First Build Session

The first implementation session begins with repository/bootstrap work, not the UI and not the LLM.

35. Create the repository structure defined in Document 14.
36. Initialize the runtime/dependency lockfiles.
37. Create the core package/module boundaries.
38. Configure formatter, linter, type checks, and tests.
39. Create the initial configuration contract.
40. Create the initial persistence schema.
41. Create the first canonical domain objects: Resource, Invariant, TrustedState, CandidateState.
42. Create the first test fixtures.
43. Make the complete bootstrap pass in CI.

Only after this foundation passes should implementation move to the next phase.

43. Final Implementation Directive

Documents 01–15 now form the complete pre-implementation engineering baseline for CloudShield AI / DAIL.

Implementation shall proceed phase-by-phase, with each phase producing testable artifacts and passing an explicit exit gate before dependent work is considered complete.

The system is not considered successful merely because it can generate Terraform or produce a visually impressive demonstration. The implementation is successful when DAIL can demonstrably take an untrusted AI-generated infrastructure repair proposal, analyze its identity and dependencies, determine affected invariants, verify the required properties, preserve an auditable evidence chain, and advance Trusted State only when the complete safety policy is satisfied.

Appendix A — Master Build Checklist

- P0 Bootstrap complete.
- P1 Domain foundation complete.
- P2 Evidence foundation complete.
- P3 Identity/dependency complete.
- P4 Impact/invalidation complete.
- P5 Verification complete.
- P6 Promotion complete.
- P7 LLM integration complete.
- P8 Experiment harness complete.
- P9 AWS/Terraform integration complete.
- P10 Hardening/E2E complete.
- Final QA report PASS.
- Final experiment report generated.
- Final demonstration repeatable.
- Project Definition of Done signed off.

Appendix B — Master Dependency Table

Phase	Depends on	Produces
P0	None	Engineering foundation.
P1	P0	Domain/state contracts.
P2	P0,P1	Evidence backbone.
P3	P1,P2	Identity + dependency.
P4	P3	Impact/invalidation.
P5	P4	Verification.
P6	P5,P2	Promotion.
P7	P6	LLM candidate generation.
P8	P6,P7	Experiment harness.
P9	P0,P3,P5,P6	AWS/Terraform integration.
P10	P1–P9	Release-ready MVP.

Appendix C — Documentation Baseline Complete

The pre-implementation documentation set is now complete:

#	Specification
01	Software Requirements Specification
02	System Architecture Specification
03	DAIL Functional Specification
04	Terraform & AWS Technical Specification
05	Domain & Data Model Specification
06	State & Invariant Lifecycle Specification
07	Identity & Dependency Specification
08	Impact & Invalidation Specification
09	Verification & Promotion Specification
10	LLM Integration Specification
11	Evidence & Logging Specification
12	Testing & QA Specification
13	Experiment Harness Specification
14	Development & DevOps Specification
15	Implementation Roadmap & Definition of Done

This document is the handoff point from architecture/specification into implementation.